Ouroboros v7.1 — Architecture & Novelty Analysis

What we are building, how it works, what problem it solves, and why the solution is new.
Companion to the *Campaign Report*; implementation evidence cited throughout is established in that document.

1 · The problem: why code-refactoring AI hits a wall

1.1 The autoregressive entropy sink

Dominant coding assistants generate tokens strictly left-to-right. For *localised completion* this works. For **macro-architectural refactoring** — method extraction, control-flow inversion, class splitting — it fails structurally: inserting even one token invalidates every downstream positional encoding, forcing full regeneration of the file tail. Cost scales linearly with file size, making real-time IDE refactoring economically impossible. Worse, the causal bias forces the model to commit early structural decisions before global context is visible — an "entropy sink" that makes non-linear mutations (reordering, splitting) statistically unavailable regardless of model size.

1.2 The diffusion alternative — and its hardware wall

Masked discrete diffusion models (MDMs) refine a fully-corrupted sequence iteratively, exposing the entire sequence state at every step: any-order generation, non-linear edits, parallel token prediction. Recent systems demonstrate the paradigm on code. But academic implementations share a fatal systems assumption: when the model requests sequence growth (`[EXPAND]`) or shrinkage (`[DELETE]`), tensors are resized mid-generation. In PyTorch this triggers O(N) reallocation, destroys cache coherency, fragments VRAM, and shatters static CUDA Graphs — the exact mechanisms that make GPU inference fast. We call this gap between algorithmic papers and silicon *"PyTorch Fantasy Land."*

1.3 Prior art leaves an intersection unoccupied

Existing strength	What it lacks
vLLM PagedAttention: paged KV memory, static graphs	Built for autoregressive decode — no any-order expansion semantics
DiffuCoder / DREAMON: diffusion for code	No memory manager that survives mid-generation growth on real hardware
Graphormer: structural bias in attention	Targets molecules/proteins; not code ASTs, not diffusion loops, not IDE latency budgets
egg / equality saturation: proof of refactor correctness	Synchronous, RAM-unbounded — unusable inside a training or serving loop as-is
Ouroboros v7.0 (prior internal spec)	Combined dynamic expansion WITH AOT ragged CUDA graphs — an intersection that is physically unrealisable on current GPUs

The thesis of v7.1: stop asking one paradigm to do everything. Separate the problem into phases and modules whose difficulty is *individually* solvable with known engineering, connect them through narrow contracts, and let a mathematical proof obligation (e-graph equivalence) — not model confidence — certify the result.

2 · The architecture: seven modules, seven named mechanisms

M1 · Pure any-order diffusion over a fixed buffer (ouroboros-core)

A masked diffusion policy operates on a **fixed L_MAX=1024-token buffer**: sequences are front-packed, unused tail slots carry `[IGNORE]`, and growth/shrinkage are *logical* operations (`insert_masks` / `logical_delete`) that never resize the physical tensor. The physical shape is invariant by test-enforced law. Training uses **Coupled-GRPO**: antithetic mask pairs ($m^1 \cup m^2 = 1$, $m^1 \cap m^2 = \varnothing$) with rewards from fast heuristic proxies only — R = 0.1·Parses + 0.3·TypeChecks + 0.6·PassesTests — because semantic verification is undecidable and must never enter the training loop (Rice's theorem).

M2 · Block-level chunking & AOT CUDA graphs (ouroboros-triton)

Production memory abandons monolithic buffers entirely: sequences live in **non-contiguous 64-token physical blocks** chained by an array-backed linked list (the block table), mirroring OS virtual memory. Growth = allocate one fresh block, link it — O(1), zero resizes, coalesced access within blocks. A single static Ahead-of-Time CUDA graph processes exactly one 64-token block; dynamic length changes merely change how many times the CPU replays it. On Grace Blackwell unified-memory platforms, capture is wrapped in a try-catch dummy-tensor warmup that forces TLB initialisation before real capture (an empirically discovered hardware trap).

M3 · 1-D RoPE + additive AST graph bias (ouroboros-core)

Prior attempts rotated embedding channels by AST depth and sibling index. That coordinate space is not orthogonal — parent/child distance becomes indistinguishable from unrelated-node distance — so it is **banned by law**. Instead: standard 1-D RoPE carries lexical order; tree-sitter computes the shortest-path-distance matrix $\varphi(i,j)$ over the AST; log-compressed scalar biases $b_{\varphi(i,j)}$ are injected additively into pre-softmax logits. Hardware efficiency of vanilla attention is preserved; topology awareness arrives as arithmetic, not architecture.

M4 · Block-sparse lexical scoping (core ↔ triton)

Bidirectional diffusion demands full attention, but dense $O(L^2)$ breaks IDE latency. Tokens are partitioned by tree-sitter spans into GLOBAL (signatures, imports, class headers) and LOCAL (function bodies). The mask enforces a strict hierarchy: locals see globals and themselves and their own scope; globals are insulated from locals (freezing the global KV-cache permanently after first pass); sibling scopes cannot bleed. When a local block expands, recomputation touches only the dirty local block.

M5 · Asynchronous e-graph congruence closure (ouroboros-dfg)

Semantic correctness is undecidable — so it is handled honestly. Both original and refactored programs lower to SSA data-flow graphs; egg equality saturation runs as a **background CPU task with hardcoded bounds** (IterationLimit 5000, TimeLimit 10 s, NodeLimit 1M, BackoffScheduler match 5000/ban 3). If roots merge: green "mathematically verified". Otherwise: yellow "passed tests, formal equivalence unproven". The green/yellow ratio is a first-class SRE metric.

M6 · Supervisor–Worker high availability (ouroboros-triton + orchestrator)

Bespoke kernels segfault — treated as a known engineering reality. An isolated C++ worker dies instantly on illegal memory access; the supervisor owns the KV-cache and block table (workers are stateless and restartable), respawns instantly, and degrades the affected request to the Phase-1 PyTorch “Safe Queue”. Blast radius: one request, not the node.

M7 · Multi-repo governance (this repository)

Three repositories enforce dependency topology (AI research / bare-metal optimisation / compiler verification). Cross-cutting them is the governance layer built during this campaign: four inviolable architectural laws validated by a model-assisted gate before any specialist writes code, post-tool crash instincts persisted to memory, and a live office visualisation of the swarm.

3 · How we are building it: the law-governed swarm

The system was not hand-written. It is being constructed by a coordinated swarm of specialist AI agents (tdd-guide, systems-engineer, rust-specialist) under an orchestration layer, powered by the ox-alpha model. The method has five enforced properties:

- **Law validation before implementation.** Every specialist intent is checked against `memory_seeds/laws.json` — the four risk-mitigations of §1 rendered machine-checkable — via a reasoning-model validator returning ALLOW/BLOCK.
- **TDD discipline.** Tests are authored before implementations across all waves; red phases were confirmed (ImportError/ModuleNotFoundError) before green.
- **Independent verification.** The orchestrator re-executes every suite itself; agent claims are accepted only after reproduction. Several "green" claims failed this audit and were repaired.
- **Honest boundaries.** Anything physically unverifiable on available hardware is documented with its exact unblock step (DEFERRED.md) rather than simulated.
- **Persistent institutional memory.** WAVELOG.md and CAMPAIGN.md record every wave, defect, and decision — the project's memory survives session turnover.

This campaign is itself the method's evidence: eight waves produced ~330 green tests, a GPU-proven kernel, and a demonstrated learning curve, across three repositories, on a free-tier model, surviving broken mirrors, poisoned DNS, and missing executables along the way.

4 · Why this is novel

> **N1 · Expansion-native paged diffusion.** PagedAttention gave AR decoding scattered KV memory; DiffuCoder showed diffusion code generation. Nobody has combined a paging memory manager with *any-order diffusion expansion* — where the model's own [EXPAND] decisions drive block allocation through a linked-list page table while a static CUDA graph replays per block. v7.1's Phase-1 primitives (insert_masks/logical_delete over a shape-invariant buffer, slot-identity block tables, chain expansion with all-or-nothing failure semantics) are the first tested step of that combination.

> **N2 · Additive AST bias as a law, not a choice.** Replacing naive 2-D RoPE with 1-D RoPE + shortest-path bias is known technique (Graphormer lineage); the novelty is enforcing it as a *machine-checked architectural law with a structural guard* — the position encoder raises on 2-D grids — plus a golden-reference oracle that pins channel-layout semantics (including the out-shuffle involution finding at $d \in \{2,4\}$, order 3 at d=8).

> **N3 · Undecidability-honest verification as telemetry.** Verification systems usually pretend to be complete. v7.1 treats Rice's theorem as a product constraint: e-graph proofs run capped and asynchronous, emit explicit Green/Yellow records, and expose the green/yellow ratio as an SRE-tunable parameter balancing proof power against warning fatigue. The yellow state is a designed outcome, not a failure.

> **N4 · Coupled-GRPO with antithetic mask pairs for refactoring.** Perfectly complementary mask pairs (disjoint, union-covering) give variance-reduced learning signal specialised to masked diffusion; fuzzy proxies

keep undecidable checks out of the loop by construction. Implemented and learning-proven (×2.0–2.8 held-out lift) at scaffold scale.

N5 · Law-governed swarm construction (the meta-novelty). The most unusual element is the build process itself: a small committee of AI specialists, each owning one repository, gated pre-write by machine-checked laws, post-hoc by independent orchestrator verification, supervised by a crash-learning hook — producing a GPU-proven kernel, a proven learning curve, and eleven caught defects across ten waves. The governance layer that will police the serving system is the same one that built it.

5 · What it solves, concretely

- **For the IDE user:** sub-200 ms structural refactoring suggestions whose correctness story is layered — tests first, formal equivalence when provable, honest warnings otherwise — instead of confident guesses.
- **For the platform engineer:** a serving layer whose failure mode is one degraded request (Safe Queue), not a crashed node; memory that provably never reallocates mid-generation.
- **For the researcher:** a working bridge from diffusion code models to real GPU memory management, with every mathematical claim pinned by tests and every hardware boundary documented with its unblock step.
- **For teams adopting agentic development:** a demonstration that architectural laws can be encoded once and enforced across every agent action — the difference between hoping agents comply and knowing they did.

6 · Evidence summary (all independently reproduced)

Claim	Evidence
Fixed-buffer diffusion loop runs with shape invariant	Algorithm-1 engine; len(buffer)==L_MAX asserted every step
Paging memory survives growth/crash cycles leak-free	Slot-identity BlockTable; demo invariants: alloc 28 == freed 28, delta 0
Triton kernel matches dense oracle	T4 execution: 120 passed/6 skipped/0 failed after three hardware-caught fixes
RoPE permutation semantics	d=8 study: un-permuted 8.3e-2 FAIL, permuted 6e-8 PASS
Policy learns EXPAND placement	Held-out lift ×2.0 (orchestrator) / ×2.54–3.38 (agent, 5 seeds); attention head +0.156 abs on unseen seeds
Refactor equivalence proofs	assoc/commute/identity Equivalent; control Unproven — via PyO3 from Python
Law enforcement	ALLOW/BLOCK verdicts recorded for every specialist intent; validator reasoning-model-aware

7 · Limitations & current boundaries

- GB10 ATS/TLB warmup path ships as logic but reproduces only on Grace Blackwell hardware.
- C++ parity execution awaits Python.h on the dev host (one apt command; scaffold + skip-guards ready).

- Real-scale GRPO training requires >4 GB VRAM; curriculum generator and trainer are complete and waiting.
- The placement-head readout is trained on synthetic gaps; transfer to production snippets is future work.

8 · Conclusion

Ouroboros v7.1 converts a mathematically sound but hardware-hostile specification into a tested, GPU-proven, honestly-bounded implementation — built by governed AI agents rather than by hand. Its novelty is not one trick but a composition: diffusion-native paged memory, law-pinned positional semantics, undecidability-honest verification, variance-reduced coupled training, and a self-constraining swarm that built it all. Each element is separately evidenced in this campaign; together they form a credible path to zero-liability, real-time code refactoring.